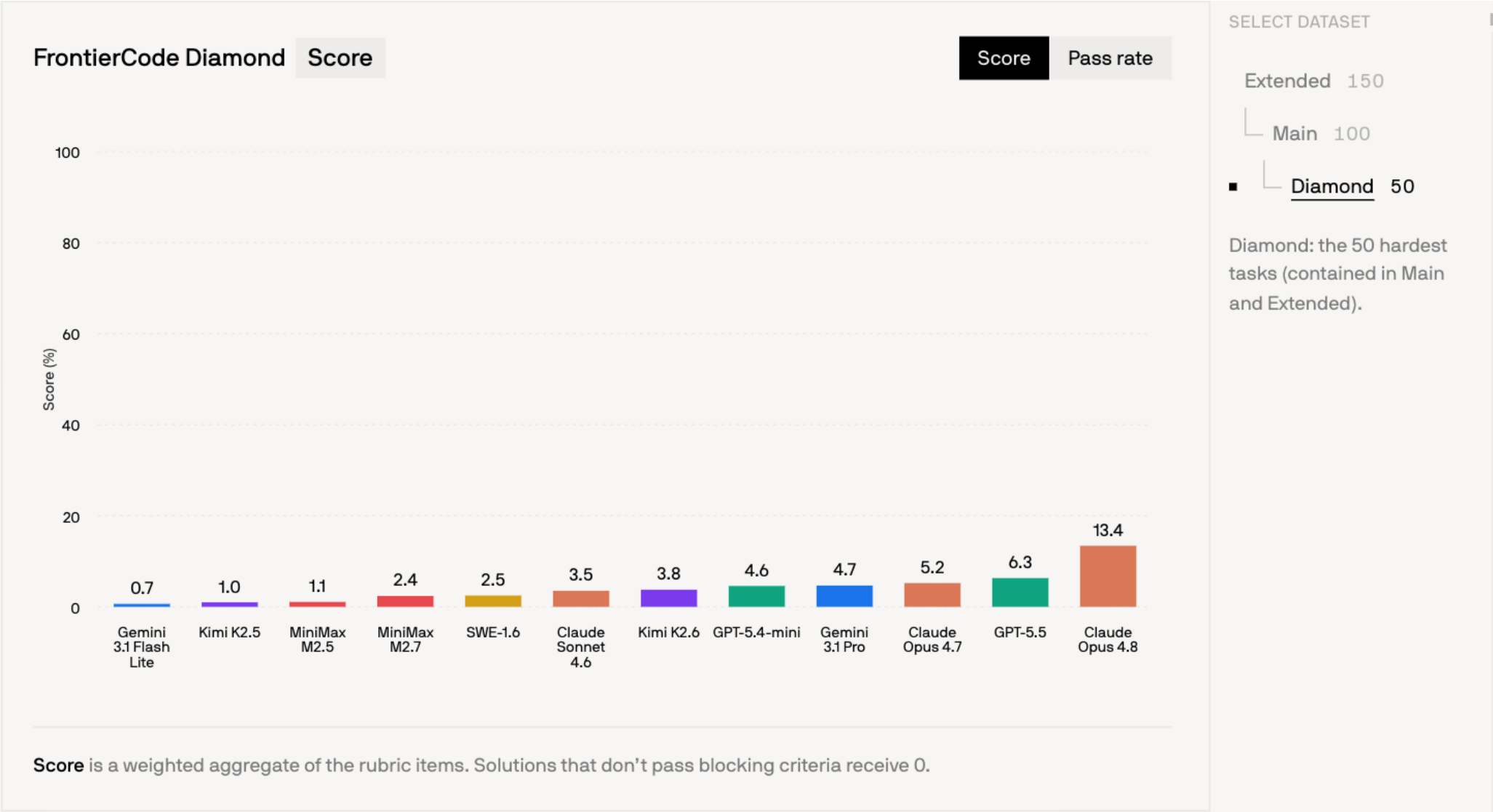




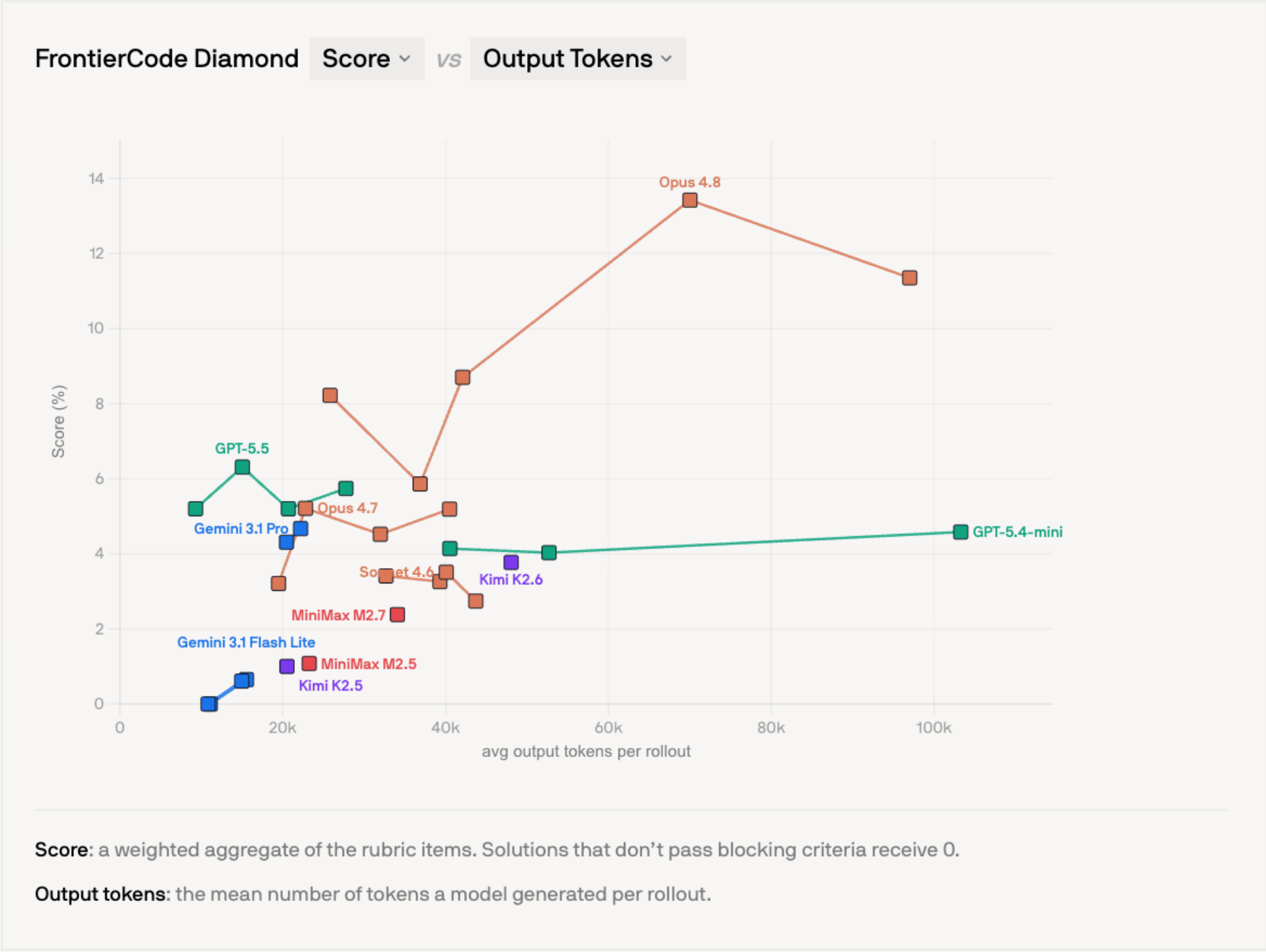
SELECT DATASET

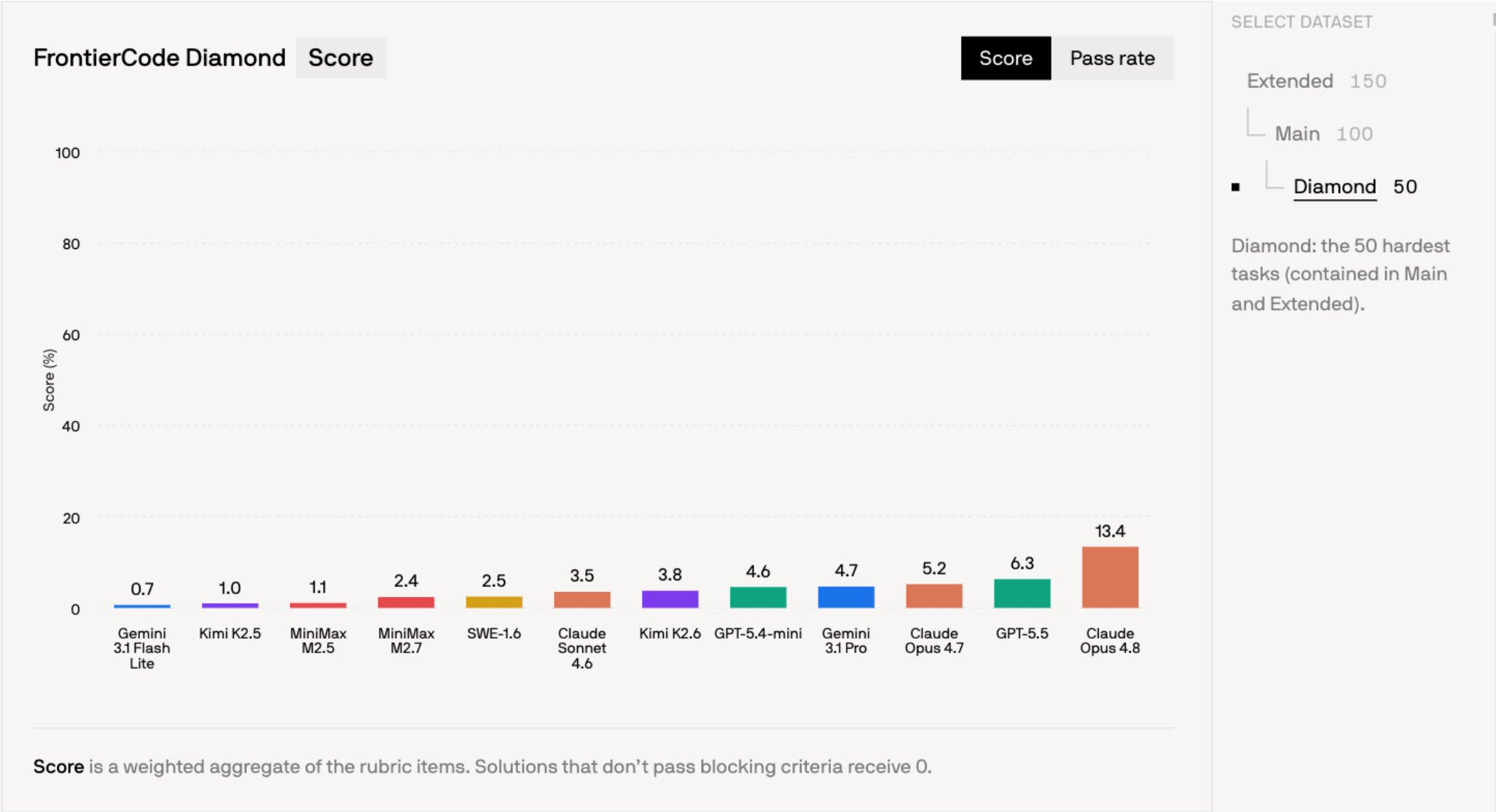
Extended 150

Main 100

Diamond 50

Diamond: the 50 hardest tasks (contained in Main and Extended).





SELECT DATASET

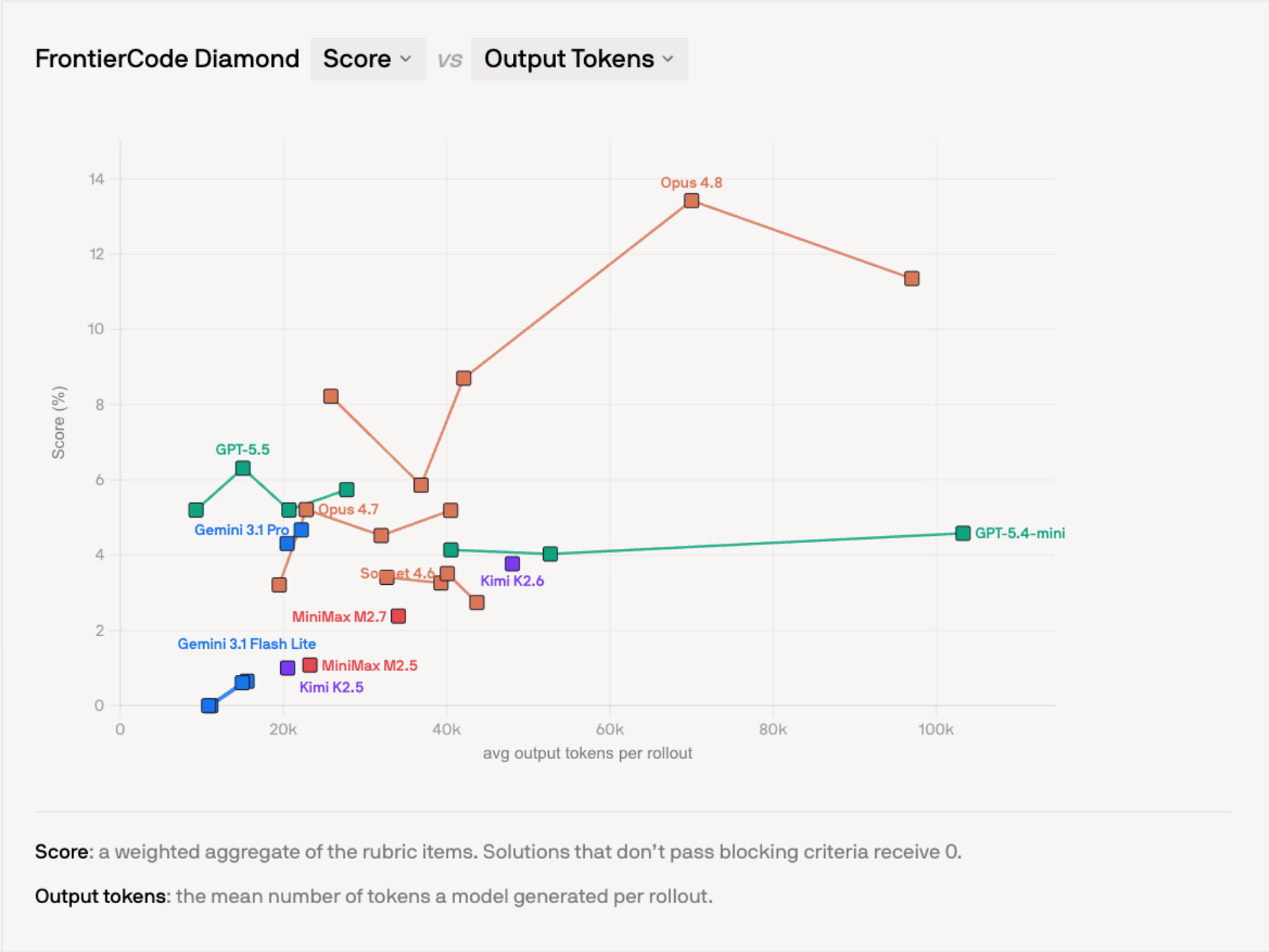
Extended 150

Main 100

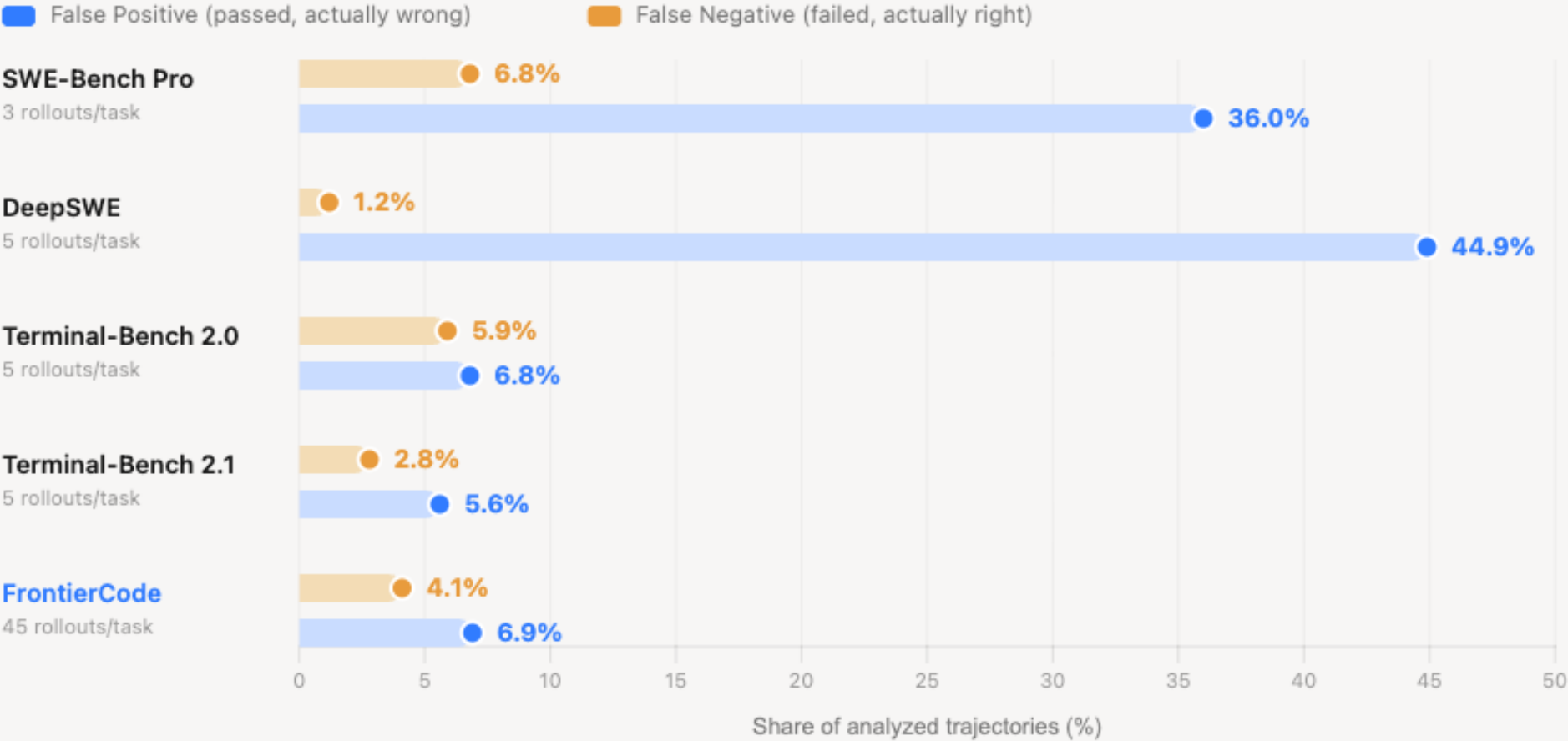
Diamond

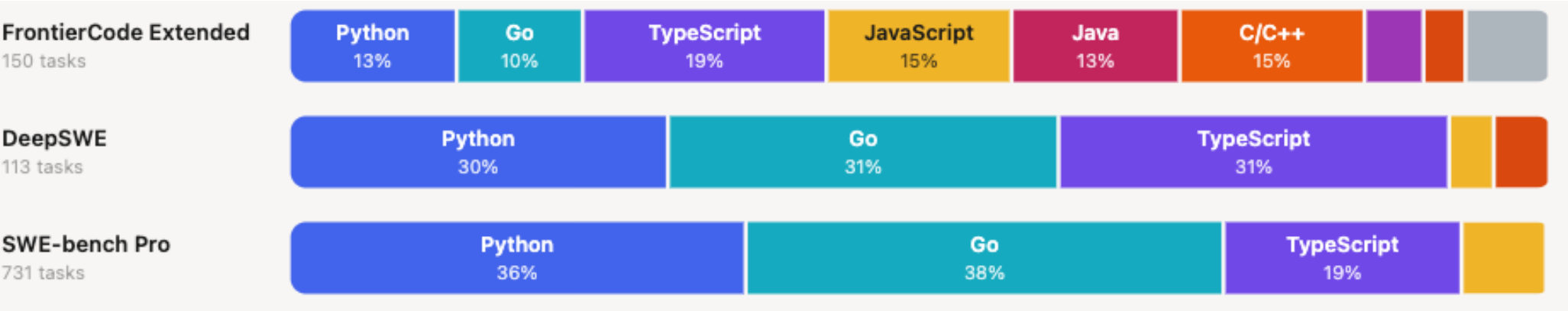
50

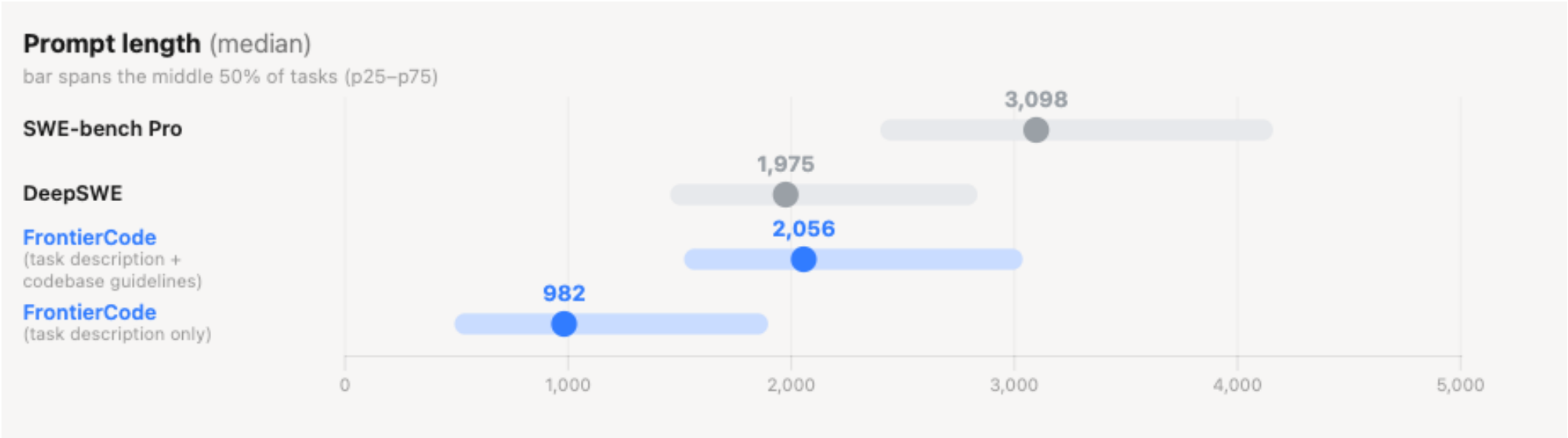
Diamond: the 50 hardest tasks (contained in Main and Extended).

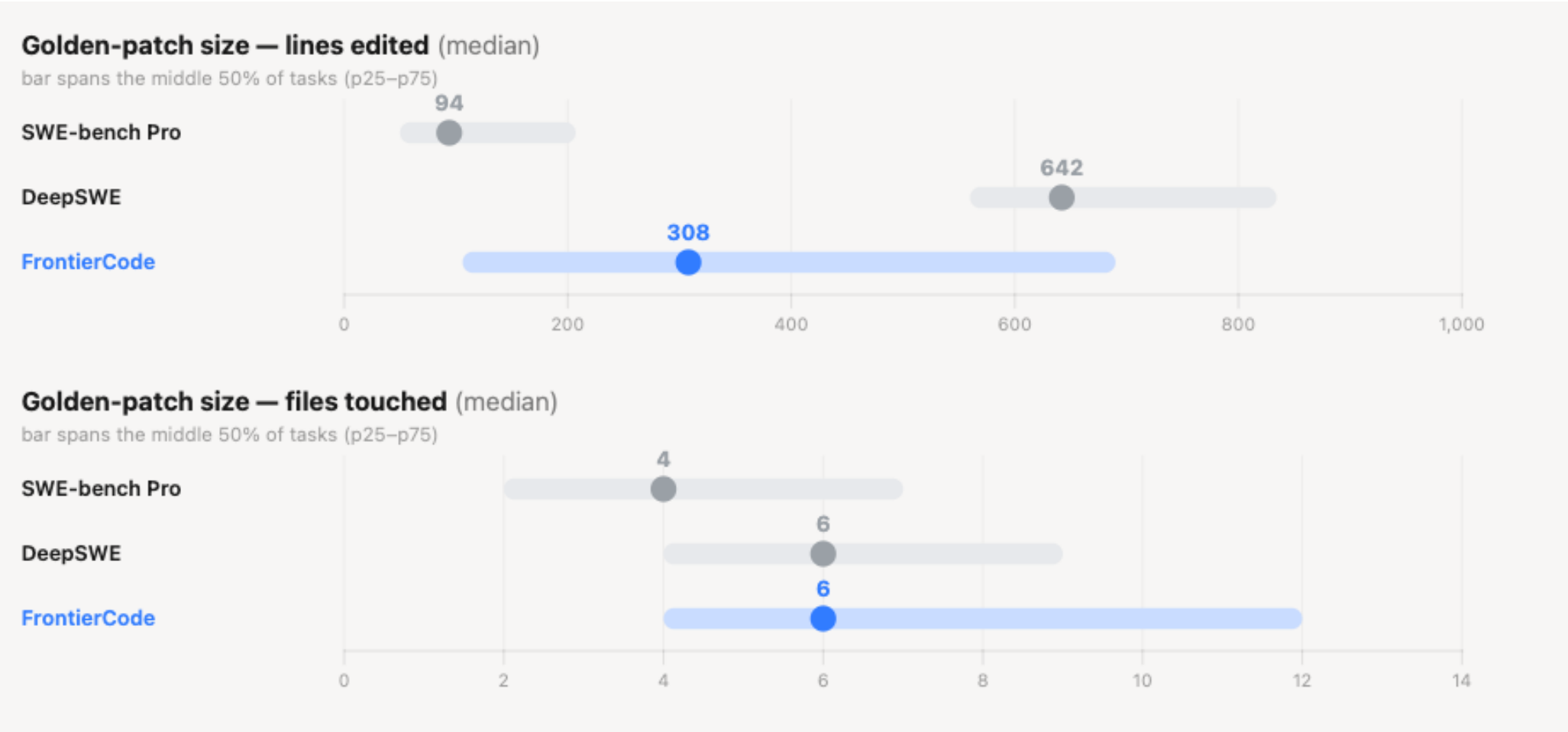


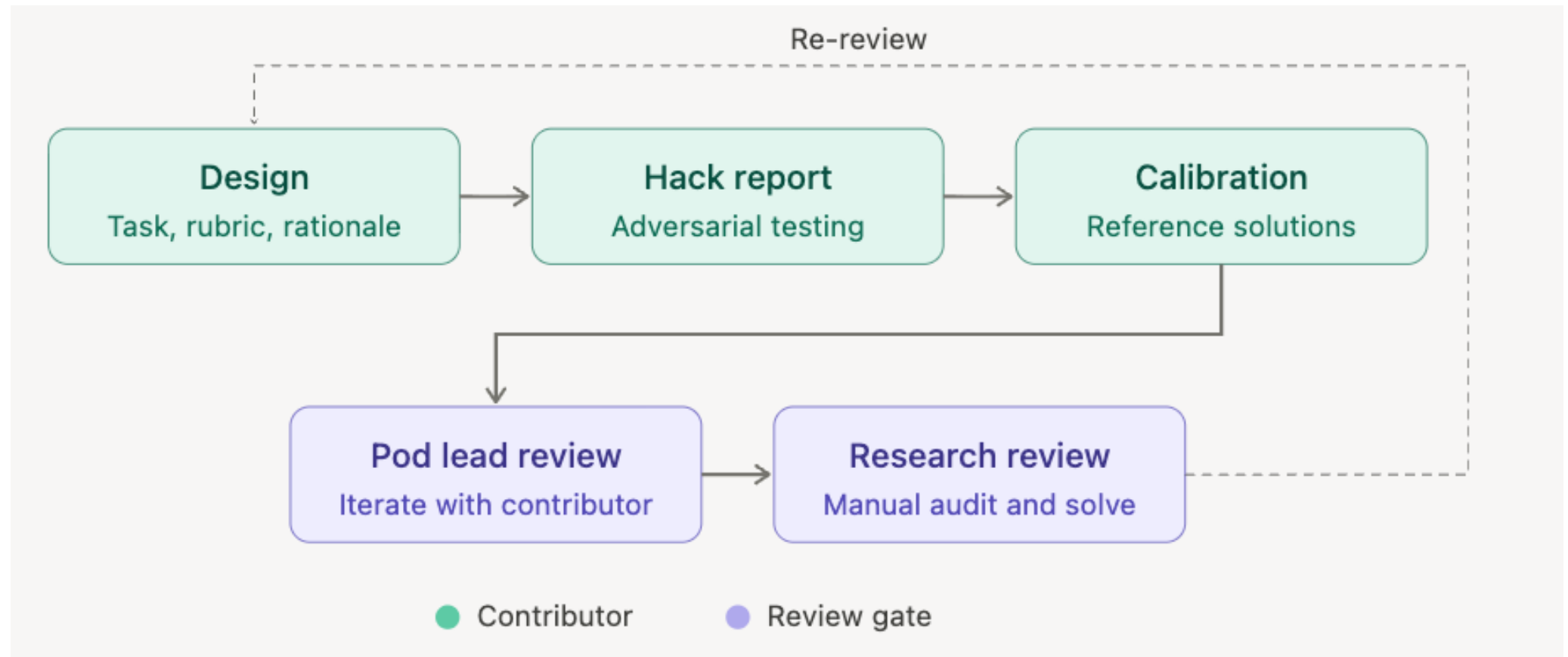
Trajectory false positive & false negative rates by benchmark











Opus 4.8 GPT-5.5

8 files +69 -22

Run again

Task description

Encapsulate all warning logs in a new `auto LOG_WARNING()` -> `std::ostream &` method in `src/logger.h` such that:

- Warnings are always printed to standard error
- Warnings are always printed, independently of `-verbose`
- The helper automatically prints the `warning:` prefix

Use this new function in every instance of `warning:` `<message>` messages throughout the codebase.

Test guidelines

Run `make` and ensure no code changes remain. If there are more code changes, then it means that the code was not formatted properly.

Unless you are sure that the code change is already covered by an existing test case, always edit or create relevant tests (in the `./test`

agent.patch

Pass · 100%

src/command_bundle.cc

+13 -11

@@

9 9 #include "command.h"

10 10 #include "configuration.h"

11 11 #include "error.h"

12 + #include "logger.h"

12 13 #include "resolver.h"

13 14 #include "utils.h"

14 15

@@ auto sourcemeta::jsonschema::bundle(const sourcemeta::core::Options &options)

35 36 .recompose

36 37

37 38 if (options.contains("without-id")) {

38 - std::cerr << "warning: You are opting in to remove schema identifiers in "

39 - "the bundled schema.\n";

40 - std::cerr << "The only legit use case of this advanced feature we know of "

41 - "is to workaround\n";

42 - std::cerr << "non-compliant JSON Schema implementations such as Visual "

43 - "Studio Code.\n";

44 - std::cerr << "Otherwise, this is not needed and may harm other use "

45 - "cases. For example,\n";

46 - std::cerr << "you will be unable to reference the resulting schema from "

47 - "other schemas\n";

48 - std::cerr << "using the --resolve/-r option.\n";

10/10 criteria met. Captured the `LOG_WARNING()` stream into a reference and routed every continuation line through it, so both multi-line warnings convert cleanly. All criteria met.

✓

Update `src/resolver.h` to use `LOG_WARNING` to print the multi-line warning in the `CustomResolver` constructor

✓

Update `src/command_bundle.cc` to use `LOG_WARNING` to print the multi-line warning in the `options.contains("without-id")` conditional

✓

Update `src/logger.h` to implement a `LOG_WARNING` function that takes no arguments and returns a non-const `std::ostream &` to standard error with a `warning:` prefix pre-printed

✓

Update `src/command_lint.cc` to use `LOG_WARNING` for the single-line warning in the `disable_lint_rules` helper

Interactive: run the FrontierCode grading pipeline against each model's output and inspect how the patch maps to rubric pass/fail.

网页视觉补充 / 2026-09-11 / 第 8 页 / 完整内容请结合正文、原图与示例数据

Opus 4.8

GPT-5.5

8 files +53 -11

Run again

Task description

Encapsulate all warning logs in a new `auto LOG_WARNING()` -> `std::ostream &` method in `src/logger.h` such that:

- Warnings are always printed to standard error
- Warnings are always printed, independently of `-verbose`
- The helper automatically prints the `warning:` prefix

Use this new function in every instance of `warning:` `<message>` messages throughout the codebase.

Test guidelines

Run `make` and ensure no code changes remain. If there are more code changes, then it means that the code was not formatted properly.

Unless you are sure that the code change is already covered by an existing test case, always edit or create relevant tests (in the `./test`

agent.patch

Fail · 24%

src/command_bundle.cc

+3 -2

@@
9 9 #include "command.h"
10 10 #include "configuration.h"
11 11 #include "error.h"
12 + #include "logger.h"
12 13 #include "resolver.h"
13 14 #include "utils.h"
14 15
@@ auto sourcemeta::jsonschema::bundle(const
sourcemeta::core::Options &options)
35 36 .recompose
()
36 37
37 38 if (options.contains("without-id")) {
38 - std::cerr << "warning: You are opting
in to remove schema identifiers in "
39 - "the bundled schema.\n";
39 + LOG_WARNING() << "You are opting in to
remove schema identifiers in "
40 + "the bundled schem
a.\n";
40 41 std::cerr << "The only legit use case
of this advanced feature we know of "
41 42 "is to workaround\n";
42 43 std::cerr << "non-compliant JSON Schem
a implementations such as Visual "

src/command_lint.cc

+2 -2

@@ static auto
disable_lint_rules(sourcemeta::core::SchemaTransformer
&bundle,

8/10 criteria met. Converted only the first line of each multi-line warning and left the continuation `std::cerr` lines untouched — baking in the assumption that `LOG_WARNING()` and `std::cerr` are the same stream. Fails both multi-line blockers.

✗ Update `src/resolver.h` to use `LOG_WARNING` to print the multi-line warning in the `CustomResolver` constructor

BLOCKER

✗ Update `src/command_bundle.cc` to use `LOG_WARNING` to print the multi-line warning in the `options.contains("without-id")` conditional

BLOCKER

✓ Update `src/logger.h` to implement a `LOG_WARNING` function that takes no arguments and returns a non-const `std::ostream &` to standard error with a `warning:` prefix pre-printed

✓ Update `src/command_lint.cc` to use

Interactive: run the FrontierCode grading pipeline against each model's output and inspect how the patch maps to rubric pass/fail.

网页视觉补充 / 2026-09-11 / 第 9 页 / 完整内容请结合正文、原图与示例数据

▼ Compare example prompts across SWE-Bench Pro, DeepSWE, and FrontierCode			Hide
SWE-Bench <code>qutebrowser/qutebrowser-4... Pro</code>	DeepSWE <code>tomlkit-toml-table-converters</code>	FrontierCode <code>jsonschema · LOG_WARNING</code>	
Title: Inability to Reliably Extract Chromium Version from ELF Binaries in Qt 6.4+ Description The mechanism for retrieving version information from <code>QtWebEngine</code> ELF binaries relies on matching a specific user-agent-like string within the <code>.rodata</code> section of the binary. This worked reliably for earlier versions of Qt, however, in Qt 6.4 and newer, the expected full user-agent string containing both <code>QtWebEngine</code> and Chromium versions is no longer present in its prior format or location. This results in failure to extract the required version metadata when the standard pattern is not found. Expected behavior The application should be able to extract both the <code>QtWebEngine</code> and Chromium version strings from ELF binaries, even when the standard concatenated format is	TOML represents nested data in three structural forms: standard header tables, inline tables, and dotted-key assignments. This feature provides bidirectional conversion between all three, preserving values and migrating comments. ▪ <code>to_inline_table</code>, <code>to_standard_table</code>, <code>to_dotted_keys</code>, <code>to_super_table</code> live in <code>tomlkit.convert</code> and are re-exported from the top-level <code>tomlkit</code> package. ▪ All conversion functions mutate <code>doc</code> in place and return the same document instance. Results satisfy <code>parse(dumps(doc))</code> round-trip integrity. ▪ <code>ConversionError</code> (<code>TOMLKitError</code> subclass) lives in <code>tomlkit.exceptions</code>. The raised exception carries a <code>key_path</code> attribute set to the requested dotted key path string. ▪ Nonexistent keys or non-table intermediates in <code>key_path</code> raise <code>ConversionError</code>. ▪ <code>to_inline_table(key_path, doc)</code>	Task description Encapsulate all warning logs in a new <code>auto LOG_WARNING() -> std::ostream &</code> method in <code>src/logger.h</code> such that: ▪ Warnings are always printed to standard error ▪ Warnings are always printed, independently of <code>--verbose</code> ▪ The helper automatically prints the <code>warning: </code> prefix Use this new function in every instance of <code>warning: <message></code> messages throughout the codebase. Test guidelines Run <code>make</code> and ensure no code changes remain. If there are more code changes, then it means that the code was not formatted properly. Unless you are sure that the code change is already covered by an existing test case, always edit or create relevant tests (in the	